Optimal Strategies for the Grocery Bot Challenge

Executive Summary

The Grocery Bot challenge presents four fundamentally different optimization problems disguised as difficulty levels. Each level scales in grid size, bot count, aisle complexity, and order size — demanding progressively more sophisticated algorithms. The core challenge is a **lifelong multi-agent pickup-and-delivery (MAPD) problem** with sequential order fulfillment, inventory capacity constraints (3 items per bot), and collision avoidance. This report breaks down the optimal approach for each difficulty, covering pathfinding, task allocation, coordination, and heuristic rules.[1][2]

Shared Foundations Across All Levels

Before examining each difficulty, several algorithmic building blocks apply universally:

Pathfinding: BFS/A* on the Grid

All levels require shortest-path computation on a 2D grid with wall obstacles. **Breadth-First Search (BFS)** is optimal for unweighted grids and runs in $O(V + E)$, making it the ideal baseline pathfinder. For larger grids or when heuristic guidance helps, **A*** with Manhattan distance heuristic provides equivalent optimality with faster average-case performance. Precomputing a **distance matrix** (BFS from every relevant cell to every other) is feasible for all grid sizes (up to 28×18 = 504 cells) and eliminates repeated pathfinding during decision-making.[3][4]

Order Pipeline Awareness

Orders are sequential with one active and one preview. A critical heuristic across all levels is **pre-picking items for the preview order** while completing the active order. Since items can be picked up before an order activates (they just can't be delivered), bots should speculatively gather preview items to minimize idle time between orders.

Inventory Management

Each bot carries at most 3 items. Non-matching items stay in inventory after drop-off, so bots should **never pick up items that don't match either the active or preview order** — dead inventory wastes capacity and rounds.

Easy (12×10, 1 Bot, 2 Aisles, 4 Item Types)

Problem Nature

This is a **single-agent Traveling Salesman Problem with Pickup and Delivery (TSPPD)** under capacity constraints. With only one bot, there is no coordination needed — the entire challenge reduces to optimal routing.[5][6]

Recommended Algorithm: Greedy Nearest-Neighbor with Lookahead

The **nearest-neighbor heuristic** is the standard approach for constrained pickup-and-delivery TSPs and performs well on small instances. For a 12×10 grid with only 4 item types and 2 aisles, the search space is small enough that even exhaustive evaluation of pickup orders is feasible.[7][8]

Optimal Strategy

1. **Precompute full BFS distance matrix** at round 0. With 120 cells, this takes negligible time.
2. **Determine needed items** from the active order (subtracting already-delivered items) and the preview order.

3. **Plan a pickup tour**: Evaluate all permutations of needed item locations (at most 4–6 items across both orders) and select the shortest route that respects the 3-item capacity. With ≤6 items, brute-force permutation ($6! = 720$) is trivial.
4. **Interleave pickups and deliveries**: If the bot has 3 items or all active-order items, route to drop-off first. Otherwise, continue picking.
5. **Pre-pick preview items** when capacity allows and when a preview item is on the path to the next active-order item.

Heuristic Rules

- Always pick up an adjacent needed item (never walk past one).
- Route to drop-off when inventory is full OR when all remaining active-order items are in inventory.
- When choosing between two equidistant items, prefer items for the active order over preview items.
- If no items are needed and inventory is empty, move toward the centroid of likely next-order item positions.

Path Execution

Use **A* or BFS** to generate the step-by-step path, then emit one move per round along that path. Recompute the plan whenever the game state changes meaningfully (item picked up, order completed, new order revealed).[4]

Medium (16×12, 3 Bots, 3 Aisles, 8 Item Types)

Problem Nature

This is a **small-team Multi-Agent Pickup and Delivery (MAPD)** problem. With 3 bots, the key challenge shifts from routing to **task allocation** — dividing items among bots to minimize total completion time (makespan).[2]

Recommended Algorithm: Hungarian Assignment + Prioritized A*

The **Hungarian Algorithm** solves the optimal assignment of bots to tasks in $O(n^3)$ time. With 3 bots and 3–5 needed items per order, construct a cost matrix where each entry is the BFS distance from bot i to item j, then assign items to bots optimally.[9][10]

For collision-free execution, use **Prioritized Planning (PP)**: assign priorities to bots (e.g., by distance to target — closer = higher priority), then plan paths sequentially where lower-priority bots treat higher-priority paths as obstacles.[11][12]

Optimal Strategy

1. **Precompute BFS distance matrix** for the 16×12 grid (192 cells — still fast).
2. **Build the assignment**: For each needed item (active + preview), compute distance from each bot. Solve with the Hungarian Algorithm to minimize total travel distance.[10]
3. **Assign at most 3 items per bot** (respecting capacity). If more items exist than bot capacity, split into pickup rounds: first round picks up 3 items each, delivers, then second round.
4. **Resolve collisions with prioritized planning**: The bot closest to completing its task gets highest priority. Other bots yield (wait or detour).[11]
5. **Designate a "delivery bot"**: When items are spread across bots, have the bot with the most active-order items head to drop-off first. Others continue gathering.

Heuristic Rules

- Avoid assigning two bots to the same item — deduplicate assignments each round.
- If a bot is idle (no assignment), route it toward the drop-off zone or toward unclaimed preview items.
- When two bots approach the same aisle from opposite ends, the one farther from its target yields by waiting.
- Re-run the Hungarian assignment every time an item is picked up or an order changes.

Collision Avoidance

With only 3 bots on a 16×12 grid, collisions are infrequent. A simple **wait-on-conflict** rule suffices: if a bot's next move would collide with another bot, it waits one round. For tighter situations, use local 2-step lookahead to find alternative moves.[13]

Hard (22×14, 5 Bots, 4 Aisles, 12 Item Types)

Problem Nature

This is a **full Multi-Agent Path Finding (MAPF) problem with task allocation**. With 5 bots and 12 item types, avoiding duplicate work and coordinating paths through narrow aisles becomes critical. The grid (308 cells) is large enough that naive approaches create bottlenecks.[14][15]

Recommended Algorithm: Conflict-Based Search (CBS) + Greedy Task Allocation

Conflict-Based Search (CBS) is a two-level algorithm optimal for MAPF. The high level maintains a constraint tree that tracks inter-agent collisions. The low level uses A* to find individual paths respecting constraints. CBS is practical for 5 agents on a 308-cell grid.[16][13]

For task allocation, use a **greedy auction mechanism**: each bot "bids" on the nearest needed item, and items are assigned to the lowest bidder. Ties are broken by inventory fullness (prefer bots with fewer items).[17][18]

Optimal Strategy

1. **Precompute BFS distances** from all item positions, bot positions, and the drop-off zone.
2. **Task allocation phase**: Compute the cost (BFS distance) for each bot–item pair. Use a greedy auction: iterate through items sorted by scarcity (fewest duplicates on the map), assign each to the closest available bot with capacity.[9]
3. **Path planning with CBS**: Feed all bot–target pairs into CBS to generate collision-free paths. With 5 agents, CBS typically solves

in milliseconds on grids of this size.[13]

4. **Rolling-horizon replanning**: Don't plan all 300 rounds at once. Use a **Rolling-Horizon Collision Resolution (RHCR)** approach — plan paths for 15–20 rounds ahead, then replan as the game state changes (items picked up, orders completed).[1]
5. **Aisle traffic management**: Designate aisles as one-directional when possible (bots enter from one end, exit from the other) to reduce head-on conflicts.

Heuristic Rules

- Never send two bots for the same item. Maintain a global "claimed items" set updated each round.
- If CBS times out (approaching the 2-second response deadline), fall back to Prioritized Planning.[12]
- Bots carrying items for the active order get higher priority than those carrying preview items.
- When a bot finishes its task, immediately re-auction remaining items.
- Use the **Token Passing** paradigm for decoupled planning: one bot plans at a time, treating other bots' committed paths as obstacles.[19][2]

Collision Avoidance

CBS inherently produces collision-free plans. However, plans may need mid-execution adjustment when new information arrives (order completion). Maintain a **reservation table** (space-time grid marking which cells are occupied at which timestep) and check it before committing moves.[16][13]

Expert (28×18, 10 Bots, 5 Aisles, 16 Item Types)

Problem Nature

This is a **large-scale swarm coordination problem** — a lifelong MAPD instance with high agent density. With 10 bots on a 504-cell grid (~2% density), congestion in aisles is the dominant bottleneck. Optimal MAPF algorithms like CBS become computationally infeasible

for 10 agents under a 2-second time constraint. Scalable, anytime algorithms are essential.[1][13]

Recommended Algorithm: MAPF-LNS (Large Neighborhood Search) + Token Passing

MAPF-LNS is the state-of-the-art anytime algorithm for large MAPF instances. It starts with a fast initial solution (via Prioritized Planning) and iteratively improves it by destroying and repairing subsets of agent paths using Large Neighborhood Search. It scales to hundreds of agents while providing near-optimal solutions.[20][21][22][23]

For task allocation, use **Token Passing with Task Swaps (TPTS)**: agents sequentially claim tasks via a shared token, and can swap tasks with other agents if doing so reduces overall cost. This prevents the "stuck agent" problem where a bot commits to a far-away task when a closer bot could handle it.[2][19]

Optimal Strategy

1. **Spatial partitioning**: Divide the 28×18 grid into zones (e.g., 2–3 bots per zone). Each zone covers ~2 aisles. Bots primarily operate within their zone, reducing inter-zone conflicts.[1]
2. **Initial plan via Prioritized Planning (PP)**: Quickly compute paths for all 10 bots using PP — process bots in priority order (closest to target first), each planning around committed higher-priority paths. This generates a feasible (if suboptimal) solution in milliseconds.[12][11]
3. **Iterative improvement with LNS**: Select a "neighborhood" of 3–5 agents whose paths could improve (e.g., those with highest delay), destroy their paths, and replan them with collision-free A*. Repeat until the 2-second deadline approaches.[23]
4. **Task allocation with TPTS**: Use token passing for task assignment — when a bot finishes a task, it takes the token, selects the best available task, and may swap with another bot if mutually beneficial.[19][2]
5. **Pipeline delivery**: Instead of having all bots pick items and then all deliver, run a **pipeline** — some bots are always picking while

others are delivering. This keeps the drop-off zone productive every round.

6. **Traffic corridors**: Establish designated paths through the store (e.g., main horizontal corridors for transit, vertical aisles for picking). Bots follow these conventions to reduce head-on conflicts naturally, similar to warehouse robot traffic management.[1]

Heuristic Rules

- **Yield hierarchy**: Bots with full inventory (heading to drop-off) have highest priority. Bots en route to pick up items yield to them. Idle bots yield to everyone.
- **Deadlock detection**: If two or more bots are mutually blocking, detect the cycle and have the lowest-priority bot wait or reroute. A simple cycle-detection check on the collision graph resolves this.
- **Stagger drop-offs**: Don't route 5 bots to the single drop-off cell simultaneously. Queue them: the closest delivers first, others wait in adjacent cells in sequence.
- **Preview prefetching**: Dedicate 1–2 bots to exclusively picking preview-order items while the rest finish the active order. This ensures zero idle time between orders.
- **Reassign on order completion**: When an order completes, immediately re-evaluate all assignments. Bots with now-irrelevant items in inventory should head to drop-off (those items may match the new active order) or continue holding them if they match the next preview.

Windowed Planning

Use **WHCA* (Windowed Hierarchical Cooperative A*)** as the low-level planner within the LNS framework. Plan only 8–12 timesteps ahead per replanning cycle. This bounds computation time and adapts to the rapidly changing state (items being picked up, orders completing).[24][25]

Algorithm Selection Summary

Difficulty	Grid	Bots	Pathfinding	Task Allocation	Collision Resolution	Key Technique
Easy	12×10	1	BFS + brute-force TSP	N/A (single bot)	N/A	Nearest-neighbor tour with lookahead
Medium	16×12	3	A*	Hungarian Algorithm[10]	Wait-on-conflict	Optimal assignment + simple deconfliction
Hard	22×14	5	A* with reservation table	Greedy auction	CBS[13]	Conflict-Based Search + rolling horizon[1]
Expert	28×18	10	WHCA*[24]	Token Passing (TPTS)[2]	MAPF-LNS[23]	Anytime LNS + spatial partitioning

---

Critical Implementation Details

Time Budget Management

Every response must arrive within 2 seconds. Budget allocation should be:

- **10% — State parsing and distance computation**: Update the BFS distance matrix incrementally (only recompute if walls change — they don't in this game, so precompute once at round 0).

- **30% — Task allocation**: Run the assignment algorithm (Hungarian, auction, or token passing depending on difficulty).
- **50% — Path planning and collision resolution**: Generate and deconflict paths.
- **10% — Safety margin**: Buffer for network latency and unexpected computation.

Dealing with Determinism

The challenge states that item placement and orders are deterministic per day. If multiple games can be played per day (with cooldowns), the first game can be used to **map out the full order sequence**. Subsequent games can pre-plan optimal tours for the known orders, dramatically improving performance.

Drop-Off Queuing

There is only one drop-off cell. With multiple bots, this becomes a critical bottleneck. The optimal pattern is a **conveyor queue**: bots line up adjacent to the drop-off cell, and as one finishes delivering, the next moves in immediately. The bot currently dropping off should always have the highest movement priority.

Inventory Optimization

With 3-item capacity and 3–6 item orders, a single bot can often fulfill an entire order in one trip (Easy/Medium). For larger teams, the strategy should minimize the number of drop-off trips:

- Prefer picking up 3 items that all match the active order before heading to drop-off.
- Avoid "partial trips" where a bot delivers only 1 item — instead, have it pick up more items first.
- Exception: if the active order needs only 1 more item, deliver it immediately to get the +5 order completion bonus.

References

1. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses - In this paper, we study the lifelong variant of MAPF, where agents are constantly engaged with new g...
2. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery ... - In this paper, we therefore study a lifelong version of the MAPF problem, called the multi-agent pic...
3. Pathfinding on a grid with breadth-first search : r/gamedev - Reddit - Using breadth-first search, I create pathmaps for every creature "alignment" on the game board. I cu...
4. [PDF] Path Finding - How to get from A to B. - How to get around obstacles in the way. - How to find the shortest possibl...
5. Heuristics for the One-Commodity Pickup-and-Delivery Traveling ... - This paper deals with a generalisation of the well-known traveling salesman problem (TSP) in which c...
6. [PDF] The Traveling Salesman Problem with Pickup and Delivery - The heuristic for separating π- and σ-inequalities uses a randomized greedy principle. ... A heurist...
7. Novel Tour Construction Heuristic for Pick-Up and Delivery Routing ... - This paper presents an adapted convex hull cheapest insertion heuristic that accounts for precedence...
8. [PDF] A hybrid ILS/VND heuristic for the One-Commodity Pickup ... - isima - A multi-start scheme is used in this heuristic: each iteration starts with an initial solution built...
9. Optimization techniques for Multi-Robot Task Allocation problems - A detailed explanation of the papers with the largest number of citations is provided. The Hungarian...
10. Hungarian algorithm - Wikipedia - The Hungarian method is a combinatorial optimization algorithm that solves the assignment problem in...
11. Searching with consistent prioritization for multi-agent path finding - We study prioritized planning for Multi-Agent Path Finding (MAPF). Existing prioritized MAPF algorit...
12. [PDF] Learning a Priority Ordering for Prioritized Planning in Multi-Agent ... - Abstract. Prioritized Planning (PP) is a fast and popular framework for solving Multi-Agent Path Fin...
13. [PDF] Multi-agent Path Planning Based on Conflict-Based Search (CBS ... - As a fundamental problem in many industrial

applications of multi-robot systems, research on Multi-A...

14. Multi-Agent Pickup and Delivery with external agents - ScienceDirect - This paper introduces a novel MAPD formulation, where team agents must solve their MAPD problem in a...
15. [PDF] A Systematic Literature Review of Multi-agent Pathfinding for Maze ... - This MAPF application is implemented in many areas that require the movement of various agents, such...
16. gloriyo/MAPF-ICBS: Multi-agent pathfinding via Conflict Based Search - ICBS is an extension of the Conflict Based Search algorithm developed for the purpose of finding opt...
17. [PDF] Task Allocation Using a Team of Robots - We present a survey of multi-robot task allocation cover- ing many problem variants and solution app...
18. [PDF] A Framework for Multi-Robot Coordination and Task Allocation - This paper proposes a novel framework, called CoMutaR (Coalition formation based on Multi- tasking R...
19. [PDF] Lifelong Multi-Agent Path Finding for Online Pickup and ... - In this section, we present first a simple decoupled MAPD algorithm, called Token Passing (TP), and ...
20. Anytime Multi-Agent Path Finding via Large Neighborhood Search - We compare our algorithm MAPF-LNS against a range of existing work and report significant gains in s...
21. Reevaluation of Large Neighborhood Search for MAPF - arXiv.org - We introduce a unified evaluation framework, implement prior methods, and conduct an extensive compa...
22. [PDF] Learning to Select Promising Initial Solutions for Large ... - In the context of anytime MAPF algorithms, Large Neighborhood Search. (LNS)-based MAPF is a promisin...
23. [PDF] Anytime Multi-Agent Path Finding via Large Neighborhood Search - On easy instances that the optimal algorithm CBS can solve within. 60s, our algorithm MAPF-LNS also ...
24. Cooperative pathfinding | Proceedings of the First AAAI Conference ... - Finally, Windowed Hierarchical Cooperative A* (WHCA*) limits the space-time search depth to a dynami...
25. (PDF) Conflict-Oriented Windowed Hierarchical Cooperative A* - CO-WHCA* improves agent coordination by focusing reservations on conflict-prone areas and reducing u...